Telegram Task Tracker

Ein Telegram-basierter Task-Tracker mit Web-Dashboard und Groq-gestützten Vorhersagen zu Mood, Stress und Workload.

Projektübersicht

Die App hilft dabei, Aufgaben direkt aus Telegram heraus zu erfassen, zu verwalten und bei Bedarf im Browser auszuwerten. Eine Aufgabe kann eine Beschreibung und ein optionales Fälligkeitsdatum enthalten. Zusätzlich kann das System eine Vorhersage zur zu erwartenden Belastung erzeugen.

Der Telegram-Bot ist der schnelle Bedienkanal. Das Web-Dashboard ist die Übersichtsschicht. Beide greifen auf dieselben Aufgaben- und Forecast-Daten zu.

Problemstellung und Nutzen

Aufgabenmanagement soll schnell bedienbar und trotzdem gut auswertbar sein. Genau hier setzt das Projekt an: Telegram ist ideal für kurze Eingaben, das Dashboard eignet sich für eine strukturierte Ansicht und für Vergleiche.

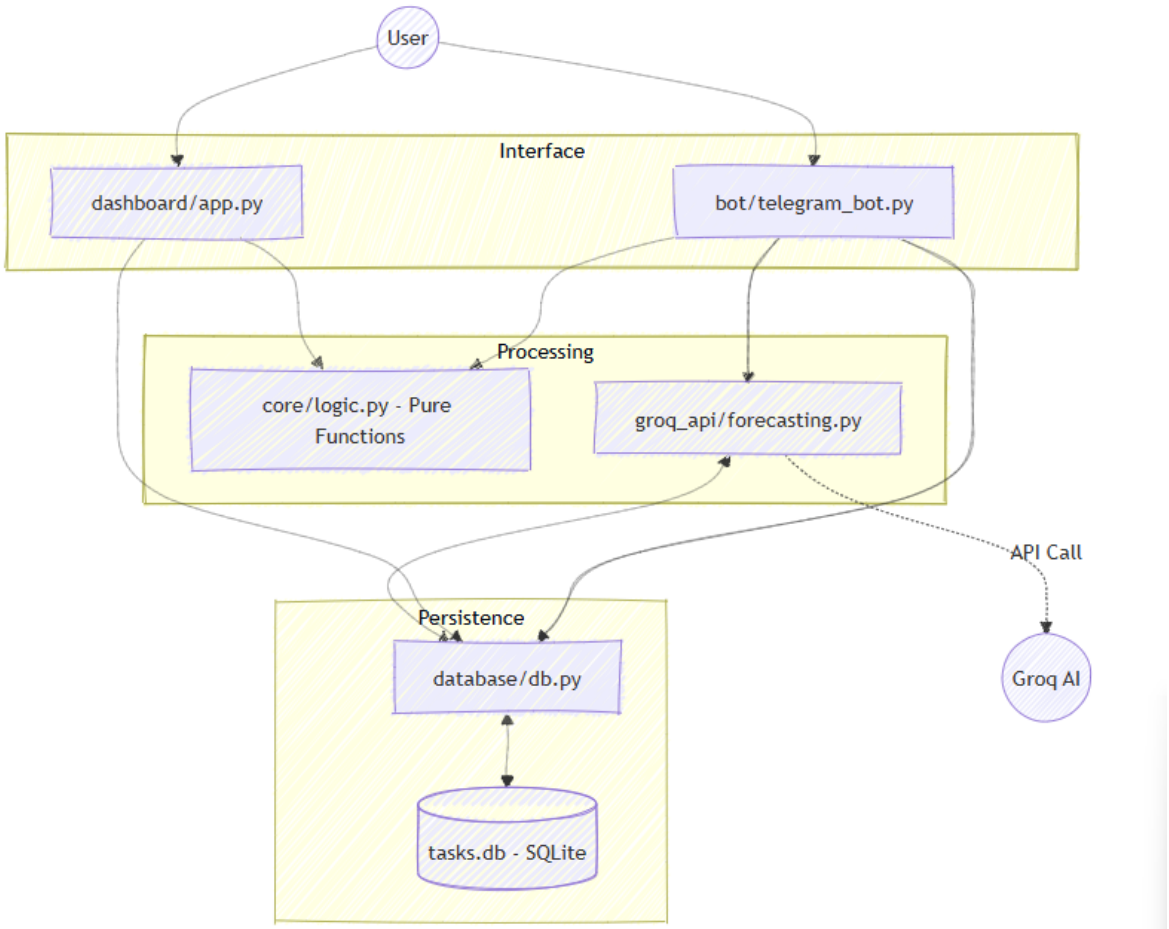
Der Nutzen entsteht durch drei Perspektiven auf dieselben Daten:

- Telegram für schnelle Kommandoeingaben
- Web-Dashboard für Übersicht und Auswertung
- Groq-Forecasts für eine einschätzbare Belastungssicht

Funktionsumfang

- Aufgaben per Telegram anlegen
- Beschreibungen und Fälligkeitsdaten erfassen
- Aufgaben auflisten
- Aufgaben als erledigt markieren
- Aufgaben löschen
- Zusatzbeschreibungen an vorhandene Aufgaben anhängen
- Forecasts für Mood, Stress und Workload für bestehende Aufgaben erzeugen
- Forecasts auf Wunsch neu berechnen
- Aufgaben und Forecasts im Dashboard anzeigen

Architektur und Datenfluss




```
graph TD
    subgraph Interface
        TB[bot/telegram_bot.py]
        WD[dashboard/app.py]
    end

    subgraph Processing
        CL[core/logic.py - Pure Functions]
        GA[groq_api/forecasting.py]
    end

    subgraph Persistence
        DB[database/db.py]
        SQL[(tasks.db - SQLite)]
    end

    User((User)) --> TB & WD
    TB & WD --> CL
    TB & WD --> DB
    TB --> GA
    GA <--> DB
```

```
    GA -.->|API Call| Groq((Groq AI))
    DB <--> SQL
```

Das Projekt ist in kleine, klar abgegrenzte Teile zerlegt:

- bot/telegram_bot.py steuert Telegram-Befehle und Dialoge
- dashboard/app.py rendert das Flask-Dashboard
- database/db.py verwaltet SQLite-Daten und Forecast-Cache
- groq_api/forecasting.py spricht mit Groq und normalisiert die Antwort
- core/logic.py enthält reine Hilfsfunktionen ohne Seiteneffekte

Der Datenfluss ist bewusst einfach gehalten:

1. Ein Benutzer sendet einen Telegram-Befehl oder nutzt das Dashboard-Formular.
2. Der Handler liest die Eingabe und ruft möglichst eine pure Hilfsfunktion auf.
3. Die Datenbank liest oder schreibt Aufgaben.
4. Die Forecast-Schicht prüft den Cache und ruft Groq nur bei Bedarf auf.
5. Das Ergebnis wird formatiert und zurückgegeben.

API-Endpunkte und Datenzugriff

Bot und Dashboard greifen auf dieselbe Datenlogik zu, um Konsistenz zu gewährleisten. Anstatt einer klassischen REST-API teilen sie sich die `database`- und `core`-Module.

- **`database/db.py`**: Stellt Funktionen wie `get_tasks()`, `add_task()` etc. bereit.
- **`core/logic.py`**: Enthält reine Funktionen zur Datenverarbeitung und -formatierung.

Ein REST-API-Äquivalent könnte so aussehen:

- `GET /tasks`: Ruft alle Aufgaben ab.
- `POST /tasks`: Erstellt eine neue Aufgabe.
- `POST /tasks/:id/forecast`: Erzeugt einen Forecast für eine bestimmte Aufgabe.

So startest du die App

Voraussetzung sind die Umgebungsvariablen `TELEGRAM_TOKEN` und `GROQ_API_KEY` in einer `.env`-Datei.

Start der kompletten Anwendung mit Telegram-Bot und Dashboard:

```
python app.py
```

Wenn du nur das Dashboard einzeln starten willst, kannst du alternativ aus dem Projektstamm heraus ausführen:

```
python -m dashboard.app
```

Datenmodell

Die SQLite-Datenbank enthält zwei zentrale Datenbereiche.

Tasks

Eine Aufgabe besteht aus:

- `id`: Eindeutiger Primärschlüssel.
- `user_id`: Telegram User-ID, um Aufgaben zuzuordnen.
- `description`: Aufgabenbeschreibung.
- `status`: Status der Aufgabe (z. B. `todo`, `in_progress`, `done`).
- `priority`: Priorität (z. B. 1-5).
- `tags`: Komma-getrennte Liste von Tags.
- `due_date`: Fälligkeitsdatum.
- `created_at`: Zeitstempel der Erstellung.
- `updated_at`: Zeitstempel der letzten Änderung.

Forecast-Cache

Forecasts werden zwischengespeichert, damit gleiche Anfragen nicht jedes Mal neu an das Modell geschickt werden müssen.

Der Cache speichert:

- normalisierten Task-Key
- originale Task-Beschreibung
- Forecast-JSON
- Zeitstempel

Funktionale Verarbeitung und Pipeline-Ansatz

Das Projekt folgt den Prinzipien des **Functional Design** (BG2). Durch den Einsatz von *Immutable Data Types* (Dicts/Tuples) und *Pure Functions* bleibt die Logik testbar und vorhersehbar. Die *Domain of Interest* (Aufgabenverwaltung) ist strikt von den Seiteneffekten (Telegram-API, DB) getrennt.
Das Projekt folgt den Prinzipien des **Functional Design** (BG2). Durch den Einsatz von *Immutable Data Types* (Dicts/Tuples) und *Pure Functions* bleibt die Logik testbar und vorhersehbar. Die *Domain of Interest* (Aufgabenverwaltung) ist strikt von den Seiteneffekten (Telegram-API, DB) getrennt.

Die Architektur ist auf der Design-Ebene deklarativ aufgebaut: Sie beschreibt die Transformation von Datenzuständen, anstatt explizite Hardware-Steuerbefehle zu geben (BE2). In `core/logic.py` werden **Algorithmen** (C1G) als finite, deterministische Abfolgen von Rechenschritten implementiert, die bei gleichem Input stets den gleichen Output liefern.
Die Architektur ist auf der Design-Ebene deklarativ aufgebaut (BE2): Das System beschreibt den Datenfluss zwischen Modulen (Input -> Logic -> Output), anstatt die exakte Abfolge von CPU-Registern oder Speicheradressen manuell zu steuern. Die Logik-Schicht implementiert **Algorithmen** (C1G) als endliche, deterministische Abfolgen von Rechenschritten.

Nachweis funktionaler Konzepte (Band C)

Um komplexe Datenverarbeitung deklarativ zu lösen (C4E), werden Lambdas und funktionale Ketten verwendet:

```
# C3G/C3F: Lambda-Ausdrücke zur Steuerung
get_desc = lambda t: t.get("description", "")
sum_ids = lambda acc, t: acc + t['id']  # Lambda mit 2 Parametern (C3F)

# C4G/C4F: Map & Filter kombiniert
pending_descriptions = map(get_desc, filter(lambda t: t["status"] != "done",
tasks))

# C4G/C4F: Reduce zur Aggregation
# C4G/C4F: Reduce zur Aggregation (Nachweis für C4G)
from functools import reduce
total_id_sum = reduce(lambda acc, t: acc + t['id'], tasks, 0)

# C2E: Closures und Currying zur Konfiguration von Filtern
def status_is(target_status):
    return lambda task: task.get("status") == target_status

done_filter = status_is("done")
done_tasks = filter(done_filter, tasks)

# C4E: Komplexe Datenverarbeitung (Gruppierung)
# C4E: Komplexe Datenverarbeitung (Gruppierung nach Status)
from itertools import groupby
grouped = {k: list(g) for k, g in groupby(sorted_tasks, key=lambda t:
t['status'])}
```

Nachweis funktionaler Konzepte (Band C)

Um komplexe Datenverarbeitung deklarativ zu lösen (C4E), werden Lambdas und funktionale Ketten verwendet:

```
# C3G/C3F: Lambda-Ausdrücke zur Steuerung
get_desc = lambda t: t.get("description", "")

# C4G/C4F: Map & Filter kombiniert
pending_descriptions = map(get_desc, filter(lambda t: t["status"] != "done",
tasks))

# C2E: Closures und Currying zur Konfiguration von Filtern
def status_is(target_status):
    return lambda task: task.get("status") == target_status

done_filter = status_is("done")
done_tasks = filter(done_filter, tasks)
```

Reine Funktionen in core/logic.py übernehmen Aufgaben wie:

- Zeilen aus der Datenbank formatieren
- Task-Listen als Text aufbauen
- Statistiken berechnen
- optionale Texte normalisieren
- Beschreibungen zusammensetzen
- Forecasts als Nachricht darstellen

Ein vereinfachter Datenfluss sieht so aus:

```
rows -> format_tasks_list -> tasks_summary -> Ausgabe
```

Der Vorteil: Der Ablauf ist leichter nachvollziehbar und die Handler bleiben deutlich kürzer.

Refactoring und Performance

Bei der Entwicklung wurden Refactoring-Techniken wie **Extract Function** (Auslagern der Formatierung in `logic.py`) und **Duplicate Removal** (Zusammenführung von Forecast-Logik) angewendet (DG1).
Bei der Entwicklung wurden Refactoring-Techniken wie **Extract Function** (Auslagern der Formatierung in `logic.py`) und **Duplicate Removal** (Zusammenführung von Forecast-Logik) angewendet (DG1). Jedes Refactoring birgt das Risiko von Regressionsfehlern (Nebenwirkungen); durch die Konzentration der Logik in Pure Functions können diese jedoch isoliert und durch Unit-Tests abgesichert werden (DE1).

Deklarative Anforderungen (Band B)

Ein Kernziel war die Umformung imperativer Abläufe in deklarative Beschreibungen (BE1):

- **Imperativ:** "Erstelle eine leere Liste. Gehe alle Zeilen durch. Wenn die ID übereinstimmt, speichere das Element und brich die Schleife ab."
- **Deklarativ:** "Finde das erste Element in der Menge der Aufgaben, dessen ID dem Suchkriterium entspricht."
    - *Umsetzung:* `next((t for t in tasks if t['id'] == search_id), None)` in `select_task_by_id`.

Die wichtigsten Performance-Massnahmen sind:

- Forecast-Caching statt wiederholter Groq-Aufrufe
- **Effiziente Datenstrukturen (DE2):** Einsatz von SQLite als persistenter Cache.
  Trade-off: SQLite bietet hohe Performance und ACID-Konformität bei minimalem Setup (Zero-Config), ist jedoch weniger für massive, verteilte Schreibzugriffe geeignet als ein dedizierter DB-Server wie PostgreSQL.
- Vermeidung unnötiger Berechnungen
- einmalige Aufbereitung der Task-Daten vor Anzeige und Statistik

Groq-AI-Forecasts

Groq wird verwendet, um folgende Werte zu prognostizieren:

- Mood
- Stress
- Workload

Die Telegram-Befehle `/forecast <id>` und `/forecast_refresh <id>` arbeiten nur mit echten Aufgaben aus der Datenbank. So wird keine Aufgabe erfunden, sondern immer die bestehende Beschreibung verwendet.

Die Forecast-Schicht ist möglichst stabil aufgebaut durch:

- einen festen Prompt
- möglichst deterministische Generationseinstellungen
- Normalisierung in ein klares JSON-Format
- Caching für wiederholte Anfragen

Mögliche Erweiterungen

- Tests für die reinen Hilfsfunktionen
- Forecast-Verlauf im Dashboard als Diagramm
- Prioritäten für Aufgaben
- Erinnerungen oder Benachrichtigungen
- Tags und Kategorien
- Export nach CSV oder JSON
- Benutzerverwaltung mit Login

Kurzes Fazit

Das Projekt zeigt, wie sich Telegram-Automation, Web-Dashboard, SQLite und AI-Forecasts in einer kompakten Produktivitäts-App kombinieren lassen. Durch die funktionale Aufteilung ist die Logik klarer, weil reine Verarbeitung und Seiteneffekte voneinander getrennt sind.

Lernziele

Band B - Anforderungen und Design beschreiben

- BG1: Imperative und deklarative Anforderungen unterscheiden.
- BF1: Den gewünschten Endzustand als Anforderung beschreiben.
- BE1: Imperative Anforderungen in deklarative Anforderungen umformen.
- BG2: Elemente des Functional Design erklären.
- BF2: Functional Design entwerfen und anwenden.

Band C - Funktionale Programmierung umsetzen

- C1F: Algorithmen in funktionale Teilstücke aufteilen.
- C1E: Funktionen in zusammenhängende Algorithmen zusammensetzen.
- C2G: Funktionen als Objekte behandeln.
- C2F: Funktionen als Argumente verwenden.

- C2E: Closures und Currying nutzen.
- C3G: Einfache Lambda-Ausdrücke schreiben.
- C3F: Mehrere Parameter mit Lambda-Ausdrücken verarbeiten.
- C3E: Lambda-Ausdrücke zur Steuerung verwenden.
- C4G: Map, Filter und Reduce einzeln anwenden.
- C4F: Map, Filter und Reduce kombiniert verwenden.
- C4E: Komplexe Datenverarbeitung lösen.

Band D - Refactoring und bestehenden Code optimieren

- DG1: Refactoring-Techniken aufzählen.
- DF1: Refactoring-Techniken anwenden.
- DE1: Auswirkungen von Refactoring einschätzen.
- DG2: Massnahmen zur Verbesserung der Leistung aufzählen.
- DF2: Vorgegebene Leistungsverbesserungen umsetzen.
- DE2: Effiziente Techniken und Datenstrukturen einsetzen.